// UNITY DEVELOPER · CREATIVE CODER · TOOL-BUILDER

Jayant *Bodse*

Three years shipping Unity titles across mobile, AR, and VR — now building a portfolio of small, *opinionated* experiments in simulation and shader work.

◆ RÉSUMÉ · 2026 — BANGALORE · INDIA

Portfolio
zxportfolio.vercel.app ↗

Email
j.bodse98@gmail.com

Phone
+91 99938 79519

LinkedIn
linkedin.com/in/jayant-bodse

Itch.io
zxdelt.itch.io

GitHub
github.com/ZxDelt

01. PROFILE

I build *small, self-contained experiences*: games, tools, simulations — each one trying to do one thing well. Equal parts game developer and tool-smith; most days I'm extending the editor or rendering pipeline before I write a single line of gameplay.

02. STACK

ENGINES

Unity 6 — primary; URP, WebGL, IL2CPP, Burst + Jobs. Also Godot, Unreal, GDevelop and Construct 3.

SPECIALTIES

Editor scripting, custom shaders, GPU simulation, cloth simulation (Magica Cloth 2 / FEM), UI Toolkit, audio reactivity, AR Foundation, Unity XR Toolkit (Meta Quest), Photon PUN multiplayer.

STRENGTHS

Scalable architecture, performance tuning, technical mentorship, cross-platform delivery, workflow automation, knowledge transfer.

LANGUAGES

C# — gameplay, editor, jobs, async pipelines. TypeScript, JavaScript, HTML/CSS, PHP, HLSL.

WEB & TOOLING

React, Next.js, Firebase, REST APIs, Git. Photoshop, Blender, Audacity for asset work.

CURRENTLY LEARNING

DOTS / ECS at scale, real-time fluid solvers, Cinemachine 3.x, advanced ShaderGraph + HLSL.

03. EDUCATION & LANGUAGES

2019 — 2022
BSc, Information Technology
Maharaja Ranjit Singh College of Professional Sciences, Indore

2017 — 2018
Higher Secondary (12th)
Marthoma H.S. School, Indore

LANGUAGES

English
fluent — daily working language

—

Hindi
native

Game Developer

Sep 2022 — Present
3 yrs 9 mon

Self-employed · Independent · Remote
Bangalore, India

Solo studio shipping a portfolio of Unity experiments end-to-end. Solved across the current six releases: **GPU water simulation** with planar reflections (Ocean Tank), **volumetric god-rays in WebGL** without compute shaders (Sanctum), **Burst-Jobs N-body particle** systems with audio routing (Particle Galaxy), a **Jos Stam stable-fluids solver** on fragment-shader passes (Inkwell), single-shader audio-reactive procedural visuals (Kaleidoscope), and a Burst-compiled cellular-automata pixel-physics sandbox (Physics Sandbox). Each release a focused study in shader work, GPU pipelines, and tactile interaction.

Core *Engineer*

Dec 2025 — Apr 2026
4 mon · contract

Kloth.me · Contract · Remote
Bangalore, India

Built the **cloth-simulation engine** behind Kloth's A.I. fit-evaluation product. Shipped four major systems: an arbitrary-cloth-to-arbitrary-body aligner with sleeve-type detection (SMPL-X UV-map filtering, per-step centroid arm-fitting); deep modifications to **Magica Cloth 2** — added a true mesh collider running inside its Burst solver, ported the entire solver to GPU compute, and built a stress/strain analysis layer driving a custom HLSL tight-folds shader; a **Unity FEM garment simulator** from first principles (CPU + GPU paths); and a **headless server-side fit pipeline** (alignment → simulation → clipping check → camera capture → A.I. color-grade).

Team *Lead*

Dec 2024 — Jan 2026
1 yr 2 mon

Mayaavi Game Studio · Full-time · Remote
Indore, India

Led a 6-person team (1 senior + 2 junior Unity, 2 frontend, 1 backend) on a third-person mobile RPG. Solved frame-rate on enemy-heavy scenes via **GPU instancing**, occlusion + frustum culling, and a single-update **Burst + Jobs** state-machine acting as a unified brain for every enemy (my first time working hands-on with Burst). Replaced Unity's Animator graph with **Animancer** for code-driven animations — smoother controller, richer combat. Built extensible enemy behaviors (dash, circle, chase, multi-attacker, fallback), a data-driven combat tool (Inspector → ScriptableObject) for weapon/animation combos, juggle system, dash VFX, enemy shaders, boss fight, and a bow-and-arrow system (homing, arrow-rain, standard).

Game Developer

Oct 2023 — Dec 2024
1 yr 3 mon

WebMobril Gaming Studioz · Full-time · On-site
Indore, India

Started my game-dev career here. Shipped client games (poker, card games, board games, simple 3D titles) and the first generation of editor tools I still use today (**Custom Logger**, **REST API async client**, **Image-to-Sprite Converter**). Built the studio's website at wmgamingstudioz.com, learned multiplayer (**Photon PUN**), integrated Unity WebGL with React for embedded play, and shipped Apple iOS / iPadOS builds end-to-end.

Game Development *Trainer*

Sep 2024 — Dec 2024
4 mon

The UpThrust · Contract · On-site
Indore, India

Taught Unity game development to a class of **6 students** while shipping client work in parallel. Debugged projects on the **Terra Game Engine**, built swipe-style mobile games, implemented security features for gambling games, and led mobile-3D performance optimization that took a Clash-of-Clans-scale Unity project from **23 fps to 60 fps**.

Game Developer

Apr 2024 — Sep 2024
6 mon

Futurristic · Contract · On-site
Indore, India

AR/VR delivery. Shipped **multi-image AR detection**, on-device **object-detection A.I. on Meta Quest**, and Quest Unity experiences (e.g. a rainy 3D grass flatland — held 60 fps against Meta's 90 fps target via **GPU instancing + occlusion + frustum culling**). Built a route-based EV-charging-rental app, explored **Apple Vision Pro** (day/night, A.I. child, spatial scan), and built Meta Quest 3/2 UI with hand-tracking + portal interactions.

05. PROJECTS

INDEPENDENT 2026 · ITCH.IO · SOLO

01 Ocean *Tank*

zxdelt.itch.io/oceantank

Independent · 2026 Unity 6, URP, WebGL, HLSL, UI Toolkit

A miniature 3D water tank with kelp, rocks, rain, sky, planar reflections — and live UI controls.

- GPU water simulation via `CustomRenderTexture` ping-pong of height/velocity buffers — click-bumps inject ripples into a damped wave equation, sampled by the surface shader for real-time displacement and foam.
- Fully procedural environment meshes — tank glass, water grid, kelp ribbons, rocks (3-octave Perlin-deformed spheres) — built at runtime, no FBX terrain.
- Manual planar reflection rig: mirrored secondary camera, oblique near-plane projection, half-res reflection RT bound to `_PlanarReflectionTex` on the water material.
- GPU rain via `Graphics.DrawMeshInstancedProcedural`, camera-tracking instances, custom additive rain shader.
- **UI Toolkit** live-controls panel: yaw/pitch/zoom, wave height & foam, reflection strength, rain intensity, cloud cover — all bound to runtime material properties.
- Shipped for WebGL — careful URP/asset-referenced material setup to dodge shader stripping; tuned camera presets and FPS counter for showcase use.

02 Sanctum

zxdelt.itch.io/sanctum

Independent · 2026 Unity 6, URP, HLSL, Volumetrics, WebGL

A small cathedral lit by colored light — stained glass, god-rays, mirror floor.

- Custom **StainedGlass** + **GodRay** shader pair: per-panel tint drives a frustum-mesh god-ray with 3D noise dust, additive blending, and distance fade — no compute shaders, WebGL-safe.
- Polished-floor shader with planar reflection RT and Fresnel-weighted sky reflection — cathedral interior reflects panels, beams, and chandelier candles.
- Procedural cathedral build: 3×3 stained-glass grid, lead/brass/silver/copper frame metals (each subtly tints adjacent panels), pointed-arch stonework, columns, pews, altar.
- Atmosphere systems: **CandleFlicker** components on 12 point lights with two-rate Perlin noise, dust ParticleSystem inside chamber bounds, drifting cumulus FBM cloud sky.
- Full URP post stack: Bloom, ACES tonemap, vignette, chromatic aberration, film grain, color grading — composed for a cinematic centered shot of altar + window.
- **WebGL optimization pass**: static batching, GPU instancing, half-res reflection RT, single-shadow directional light, Brotli + heavy code stripping for a small build.

03 Particle *Galaxy*

zxdelt.itch.io/particle-galaxy

Independent · 2026 Unity 6, Burst + Jobs + Mathematics, WebGL

N-body lite — thousands of particles orbit a gravity well, painting glowing trails on a Texture2D.

- Burst-compiled `IJob` sim: 5,000+ particles integrate Newtonian gravity + damping every frame on persistent `NativeArray`s — zero allocations after Start.
- Trails painted via second Burst job that fades the entire pixel buffer per-frame and additively splats Gaussian particle dots, then uploads via `SetPixelData`.
- **Audio-reactive overhaul**: tab-capture audio bridge with bass/mid/treble routing matrix — each band drives a different physics term (gravity / spin / trail-fade / pulse) at user-configurable strength.
- Beat-pulse explode + snap-back with a brief centerForce boost; per-channel fade biasing produces Fire and Electric trail moods.
- Mouse drag = gravity well (LMB attract, RMB repel); R re-randomizes; E radial explode; sliders on UI Toolkit panel for live tuning.
- Music-mode preset auto-engages on tab-capture (bass drives Spin, mid drives Gravity, treble drives TrailFade); idle preset stays subtle. Smooth transitions either way.

04 Inkwell

zxdelt.itch.io/inkwell

Independent · 2026 Unity 6, URP, HLSL, WebGL, Stable Fluids

Jos Stam's 2003 stable-fluids solver running real-time in WebGL. Mouse paints, ink swirls.

- Full GPU fluid pipeline — `Graphics.Blit` ping-pong on RenderTextures with `RGHalf` velocity, `RHalf` pressure, `ARGBHalf` dye — formats picked for universal WebGL2/WebGPU support.
- ~60 fragment passes per frame on a 256×256 grid: add force, optional buoyancy, optional reactive ingredients, Jacobi diffuse (~20), divergence + Poisson pressure (~30), gradient subtract, semi-Lagrangian advection.
- Multi-color dye injection with brush picker, reflective velocity walls, zero-Neumann dye boundary; fluid demonstrably incompressible at default 30 pressure iterations.
- Five hand-tuned **presets** (Marble, Smoke, Galaxy, Reactive, Calm) — each a coherent feel with viscosity, dissipation, force, and bloom set in concert.
- Audio reactivity via mic energy → per-route modulation of force / dye rate / viscosity (architecture ported from Kaleidoscope).
- Optional thermal mode (warm dye rises) and reactive ingredients (R+G yields Y, R+B yields M, G+B yields C) toggleable from UI Toolkit panel.

05 Kaleidoscope

zxdelt.itch.io/kaleidoscope

Independent · 2026 Unity 6, URP, HLSL, Audio Reactivity

Audio-reactive procedural visuals — live music drives the geometry and color fields.

- Single-shader procedural composition layered from **ribbons**, **pulse rings**, **radial spokes**, and **shimmer noise** — every layer count, frequency, amplitude, glow, and color spread is a runtime knob.
- UI-driven base values + **AudioReactor** overlay: each property has a "last UI value" that the reactor reads, layers audio modulation on top, then writes to the live material — clean restore when a route is disabled.
- Configurable routing matrix — bass / mid / treble each pick which shader property to drive, mutually exclusive selection prevents fighting modulators.
- Optional user texture upload (PNG/JPG byte loader) feeds the kaleidoscope center; toggleable between procedural and texture modes mid-show.
- Hue control with primary/secondary HSV poles, hue-shift speed, color spread, and background darkness — shader applies HSV palette mapping after composition.
- Final polish via per-shader saturation / brightness / contrast / vignette so output is grading-correct without an extra post pass.

06 Physics *Sandbox*

zxdelt.itch.io/physics-sandbox

Independent · 2026 Unity 6, Burst + Jobs, WebGL, Cellular Automata

Noita / Sandspiel-style 2D pixel physics — sand, water, fire, smoke, electricity.

- Burst-compiled cellular automata solver on a 256×256 grid — single `IJob` per frame (CA can't trivially parallelize without race conditions), persistent NativeArrays, ~96KB working set.
- Multi-pass solver: bottom-up (sand, water with multi-cell horizontal flow), top-down (fire, smoke, steam rising), any-order (spark / iron / water electricity propagation guarded by a `processed` byte buffer).
- **Element chemistry**: fire+water→steam, fire+wood→ignite, fire→smoke→air decay chain, steam condensation, sand-through-water displacement, water + spark electrical conduction.
- Per-cell color jitter at paint time + dynamic colors interpolated from `life` ticks for fire / smoke / spark / energized iron — gives natural visual variation with no extra art pass.
- UI Toolkit element grid (4×2), tools row, brush slider; keyboard shortcuts `1-7,0`, `[/]` brush size, `C` clear, `E` explosion.
- Shipped to WebGL ~11 MB after Brotli — solved real shader-stripping and `Collider.Raycast()` UV bugs along the way.

PROFESSIONAL 2022 — 2024 · CLIENT · STUDIO

07 Ludo *Multiplayer*

Webmobril Gaming Studioz
Unity · Photon PUN · Mobile

Professional · 2022–23 Photon PUN, Networking, Mobile UI

A real-time 4-player networked Ludo with full lobby, reconnect, and anti-cheat.

- Implemented full Photon PUN networking layer: room/lobby creation, matchmaking, turn synchronization, RPC piece-move events with deterministic dice resolution.
- **Reconnect logic** for dropped connections — late-joiner re-syncs board state, turn-owner, and dice history without recreating the room.
- Server-side validation of every move against turn state and dice value — anti-cheat layer rejected illegal client moves and surfaced them as soft errors.
- Built emoji chat, real-time reactions, and round-end celebration overlays to drive retention; tuned animation timing with dynamic UI transitions.
- Designed match flow UI from cold-start: matchmaking spinner → lobby fill → countdown → game → result screen, with consistent neon-mobile visual language.

08 Meta Quest *VR Sandbox*

Futurristic Business Solutions
Unity XR Toolkit · Meta Quest

Professional · 2024 XR Toolkit, 6DoF, Hand Tracking, Haptics

A VR object-interaction playground for Meta Quest — grab, throw, examine, pulse haptics.

- Built on Unity XR Toolkit: hand-tracking input, controller fallback, physics-based grab / throw / examine with object-lifecycle management.
- Implemented **snap-grab** and **distance-grab** mechanics for natural reach; tuned spring-and-damper forces so object feel held weight without jitter.
- Per-interaction haptic feedback patterns (impact, hold, release) wired into Quest controller pulse APIs; matched audio cues for each.
- Profiled and tuned for stable **6DoF** tracking under load — kept frame budget headroom for additional scene content.
- Authored cross-platform build setup (Quest 2 / 3 / Pro) with platform-specific input action maps and graphics tier overrides.

09 AR *Location* Quest System

Futurristic Business Solutions
Unity · AR Foundation · Mapbox

Professional · 2024 AR Foundation, Mapbox SDK, GPS, Geofencing

GPS-anchored AR experiences — virtual content placed at real-world coordinates.

- AR Foundation + Mapbox SDK integration: rendered route markers, location pins, and quest objectives on a live AR camera feed.
- Real-time GPS polling + **geofencing** to activate AR triggers at precise lat/long radii; story flags persisted across app sessions.
- Tuned AR session and GPS update frequencies for low battery drain — adaptive polling slowed when stationary, sped up when moving.
- Crafted a story-driven quest system with multi-stage objectives, dialog overlays, and audio narration tied to physical-world checkpoints.
- Cross-platform Android/iOS shipping with platform-specific permission flows for Camera, Location, and Mic.

10 Custom Logger Unity *Tool*

Internal — Editor extension
Unity · Editor Scripting · IMGUI

Professional · 2024 Editor Scripting, QA Tooling

An in-editor debug log tool — filtered, color-coded, exportable.

- Custom **EditorWindow** capturing runtime log entries with type, timestamp, stack trace, and tag — replaced ad-hoc `Debug.Log` sprawl.
- Advanced filtering UI: include/exclude by log type (INFO/WARN/ERROR), regex search, tag toggles, time-window slider — non-destructive (filter view, original log preserved).
- Pause / resume capture, in-place tail-follow, jump-to-source on click; integrated with Unity's native console without overriding it.
- Export current view as JSON or CSV for QA hand-off; sessions persist between play-mode runs for cross-session debugging.
- Reduced QA-hand-off friction across the team — reproduction logs ship with every bug ticket.

11

AR *Coloring*

Client project · 2023
Unity · AR Foundation · Mobile

Professional · 2023 AR Foundation, Marker Detection, iOS & Android

An AR coloring book — children paint, the camera detects color, 3D models come alive on top.

- Built an AR coloring experience on Unity + AR Foundation that detects colored images on physical paper and renders animated 3D models aligned to user drawings.
- Implemented the **marker detection** system — accurate, drift-free alignment of 3D content with hand-colored artwork in real-time.
- Sampled color data from the marker to drive both the model's albedo / texture and triggered animation states + sound feedback.
- Cross-platform Android & iOS shipping with optimized AR tracking under varied lighting; tuned camera resolution & permission flows for low-end devices.
- Collaborated with designers on child-friendly UI, scan-progress feedback, and interactive learning prompts that gated the experience for early readers.

12

Multi-Image AR *Detection*

Client project · 2023–24
Unity · AR Foundation · Mobile

Professional · 2023–24 AR Foundation, Image Targets, Storytelling

Educational AR — many image targets, many concurrent reactions.

- Built an AR Foundation app capable of detecting and responding to multiple image targets **simultaneously** — independent state per target, no thrashing on overlap.
- Each detected image triggered a unique payload: 3D animation, audio clip, or video overlay tied to specific learning content.
- Tuned tracking stability under low-light and angled scanning conditions; recovered cleanly when targets left the frame and re-entered.
- Managed real-time content swapping with **state preservation** — re-detecting an image resumed its prior progress instead of restarting.
- Designed an interactive UI for scanning progress, success status, and on-screen instructions — built for shared classroom use.

13

REST API Async *Client*

Internal — Reusable library
Unity · C# · async/await

Professional · 2024 HttpClient, Token Auth, JSON, Retries

A drop-in HTTP layer for Unity — non-blocking, typed, instrumented.

- Implemented a Unity **REST API client** built on `HttpClient` + async/await — non-blocking web communication, no main-thread stalls.
- Full GET / POST / PUT / DELETE coverage powering real-time leaderboards, inventory sync, and player data exchange.
- Reusable service layer with JSON parsing and robust error handling — automatic retry on network timeouts, structured exceptions for callers.
- Token-based auth: header injection, refresh flow, and a clean login/authentication workflow; secrets isolated from gameplay code.
- Local interaction logging for debugging and network profiling during QA — every request/response captured with timing and status.

14

Image-to-Sprite *Converter*

Internal — Editor extension
Unity · Editor Scripting · Sprite Atlas

Professional · 2024 Editor Tool, Asset Pipeline, 2D

Bulk-convert textures into Unity-ready 2D sprites with one drop.

- Designed an in-editor utility to bulk convert raw textures into Unity-ready 2D sprites — custom **pivot**, **border**, and compression settings exposed per-asset or batch.
- Drag-and-drop interface for multiple images at once; per-file overrides applied on the fly without leaving the inspector.
- Automated slicing and sprite generation for 2D projects — replaced ~30-click manual import flow with a single drop.
- File-renaming rules + auto-folder placement on import enforced consistent project structure across the team.
- Sprite Atlas–compatible output for runtime optimization; sprites bundle into atlases without further configuration.

15

Folder & Hierarchy *Setup Tool*

Internal — Editor extension
Unity · Editor Scripting · JSON config

Professional · 2024 Editor Tool, Project Templates, Onboarding

One-click project scaffolding — folders, scenes, and hierarchy from a template.

- Custom Unity editor script that auto-creates project folder trees and default scene hierarchy structures from a single menu item.
- Multiple **profile templates** — "2D Project", "VR Project", "Multiplayer" — each standardizing a different studio workflow.
- One-click setup for art, scripts, prefabs, audio, UI, and documentation folders — with consistent naming conventions enforced on creation.
- Cut team onboarding time on new repos — eliminated manual setup error and saved a sprint of dev-environment friction per joiner.
- Tool config lives in **JSON** files — teams extend templates without touching code; new profiles ship as data, not commits.

KLOTH.ME CONTRACT · DEC 2025 — APR 2026

16

Headless Fit *Evaluation Pipeline*

Kloth.me · Flagship
Unity · Server-side · A.I. Post

Kloth.me · 2025–26 Cloth Solver, Headless Capture, Server Pipeline

A headless server takes a body + garment + size — outputs a photoreal render and a yes/no fit verdict.

- Designed and shipped the end-to-end **headless Unity pipeline** running on the Kloth build server: garment FBX + body FBX in, photoreal PNG and a `fit_suitable` verdict out — no editor, no human in the loop.
- Stage chain: **garment-body alignment → arm/sleeve alignment → cloth simulation → clipping checks → headless camera capture → A.I. color-grading post**, each stage a swap-in module behind a stable contract.
- Per-call clipping detection — measures garment-to-body interpenetration after the simulation settles, flags problem regions before the render is shipped.
- Authored the cloth-solver core (atop modified Magica Cloth 2) and the alignment math; orchestrated simulation step counts, settle-frame detection, and capture timing.
- A.I. post-process pass takes the raw simulation render and returns a color-graded final image used directly in the consumer app.
- Replaced studio shoots — same body × same garment × every size, parallelized across a worker pool with predictable per-job budget.

17

Magica Cloth 2 — *Solver Extensions & GPU Port*

Kloth.me · Engine work
Unity · Burst · Compute Shaders · HLSL

Kloth.me · 2025–26 Burst, Compute, Mesh Collider, Stress / Strain

Solver-level extensions to Magica Cloth 2 — colliders, full GPU port, and a stress / strain visual layer.

- Built a **mini-sphere primitive collider grid** using Magica Cloth 2's native sphere primitive at micro-scale, placed along body boundaries so cloth never punched through — cheap, dense, faster to ship than mesh colliders.
- Upgraded that to a **true mesh collider** running inside Magica's Burst-compiled solver — required modifying Magica's solver files directly to honor mesh-vs-particle queries every substep without breaking Burst safety.
- Authored a full **GPU port** of the Magica Cloth 2 solver — moved constraint solving and integration off CPU/Burst onto compute shaders, preserved the existing constraint API so existing rigs ran unchanged.
- Built a **stress & strain analysis layer** — per-vertex stretch/shear computed each tick from solver state and exposed as a vertex stream the renderer reads.
- Wrote a custom **HLSL shader** consuming the stress/strain stream to produce physically-motivated **tight folds** — wrinkles bunch where the cloth is actually under load.
- Same stress map fed back into the fit evaluator as a region-wise "tightness score" — red zones flag too-tight fits before the render even runs.

18 Any-Cloth Any-Body *Aligner*

Kloth.me · Math + Geometry
Unity · SMPL-X · Bone Proxy

Kloth.me · 2025–26 Mesh Alignment, Sleeve Detection, Centroid Tracking

Geometric alignment of arbitrary garments onto arbitrary bodies — and the hardest part: sleeves.

- Built a system that takes any garment mesh + any body mesh and produces a draping-ready initial pose — translate, scale, and bone-proxy align before simulation begins.
- **Sleeve-type classification** from raw garment vertices — short / cap / long / sleeveless — detected before alignment so the aligner picks the correct strategy per garment.
- **Per-step centroid arm alignment** — for each sleeve segment, recompute centroid every iteration, fit it to the body's arm bone proxy, repeat until convergence; the hardest math in the project and the system's hardest-won win.
- Used the **SMPL-X UV map** as a body-region prior — projected garment vertices against body UV regions to filter out non-sleeve geometry (collar, torso, hem) so the sleeve solver only operated on sleeve verts.
- Bone-proxy driven binding — every garment vertex is weighted against its nearest body bone proxy, giving correct pose-tracking once draped.
- Robust to mesh density, scale, and topology — same code path for a t-shirt, a hoodie, and a long-sleeve dress.

19 Unity FEM *Garment Sim Plugin*

Kloth.me · R&D
Unity · CPU + GPU paths · HLSL

Kloth.me · 2025–26 FEM, Continuum Mechanics, Compute

A second cloth solver from first principles — Finite Element Method, CPU and GPU paths.

- Authored a Unity-native garment simulator on the **Finite Element Method** — discretizes the garment into triangular elements, computes per-element strain/stress, integrates internal forces every step.
- **FEM in two lines**: each triangle's **deformation gradient** F measures how the current shape differs from rest; **Green-Lagrange strain** E = (transpose(F) * F - I) / 2 quantifies stretch + shear; **stress** S = lambda * trace(E) * I + 2 * mu * E (St. Venant-Kirchhoff linear elasticity) gives the in-plane force, with discrete-bending energy added for out-of-plane folds.
- Two fully separate code paths — **CPU-only** (reference, debuggable) and **GPU-only** (production target) — same constraints, same convergence, different perf envelopes.
- Solver assembles per-element forces into a global vector, integrates with semi-implicit Euler each substep, applies position-based constraints for non-stretchy fabrics.
- Stress / strain falls out for free from the FEM formulation — same data plugged directly into the stress-shader pipeline used for Magica.
- Shipped as a half-finished plugin; GPU path was the strongest version and reached working real-time simulation before scope was deferred when the Magica route matched product needs sooner.